

# TP PoC Crypto

Niveau : 5eme année - développeur

Organisation : Individuel ou binôme

## 1 Contexte

Vous devez choisir l'un des 12 sujets lié à la cryptographie proposés ici.

Pour ce sujet, vous devez réaliser et documenter une preuve de concept (PoC)

C'est a dire un programme qui met en œuvre le principe cryptographique étudié.

Votre programme et son installation doivent être abondamment commenté.

### 1.1 Langages autorisés

Vous pouvez utiliser, pour votre code, un ou plusieurs des langages suivant :

- Compilés
  - C
  - C++
  - Rust
  - Go
- Interprétés
  - Python
  - PHP
  - JavaScript ou variantes (NodeJS, React, ...)
  - Ruby

### 1.2 Livrable

Livrable : **Un dépôt Git**

Si le dépôt est public : Me donner l'URL par MP Discord ou par mail ([hugues.levasseur@arrobe.fr](mailto:hugues.levasseur@arrobe.fr))

Si le dépôt est privé : M'inviter avec des droits suffisants pour que je voie le code

- Gitlab : <https://gitlab.com/arrobe>
- Github : <https://github.com/ArrobeHugues>

Le README.md doit :

- Donner vos noms en clair
- Rappeler le sujet
- Expliquer comment installer le logiciel & les éventuelles dépendances
- Expliquer comment utiliser le logiciel & en quoi la PoC prouve quelque chose
- ... répondre aux questions posés dans le TP

### 1.3 Date de rendu

TP à rendre avant le dimanche 19 juillet 2026, minuit

### 1.4 Barème

#### 1.4.1 Difficulté

Certains sujets sont plus difficiles que d'autres

J'ai fait 3 catégories

- Très difficile (Vous serez noté sur 20)
- Difficile (vous serez noté sur 18)
- Normal (Vous serez noté sur 16)

### 1.4.2 Originalité

Bonus de +1 si vous êtes le seul à traiter ce sujet là

### 1.4.3 Binôme

En cas de binôme, les 2 doivent contribuer autant au dépôt Git

Si les contributions sont trop inégale la note sera dégradée

Vous **ne** pouvez **pas** utiliser l'excuse « On a tout fait ensemble depuis un seul PC »

### 1.4.4 I.A.

Vous pouvez utiliser l'IA, bien sur

Mais ce doit être une aide, pas un générateur de réponse

En cas de code « sorti directement du cul de Claude », la note sera dégradée

## 1.5 Index

### 1.5.1 Très difficile

- Sujet 3 : Forge de signature RSA par faute
- Sujet 5 : Attaque par réutilisation de nonce sur AES-GCM
- Sujet 6 : Implémentation et cassage de LFSR et générateurs pseudo-aléatoires faibles
- Sujet 8 : Chiffrement homomorphe simplifié

### 1.5.2 Difficile

- Sujet 2 : Attaque par Padding Oracle sur AES-CBC
- Sujet 7 : SSLStrip : déclassement HTTPS vers HTTP
- Sujet 10 : Implémentation d'un protocole d'authentification Zero-Knowledge
- Sujet 11 : Certificate Pinning et attaque MITM sur application mobile

### 1.5.3 Normal

- Sujet 1 : Attaque temporelle
- Sujet 9 : Attaque par dictionnaire et cassage de hash avec règles
- Sujet 12 : Vulnérabilité JWT

## 2 Les sujets

### 2.1 Sujet 1 : Attaque temporelle

#### 2.1.1 Classification

- Difficulté : Normale
- Catégorie : Algorithmie

#### 2.1.2 Contexte

Certaines implémentations naïves de comparaisons de secrets (mots de passe, tokens, MAC) sont vulnérables à des attaques dites attaques temporelles : la durée de traitement varie selon les données en entrée et révèle de l'information sur la valeur secrète.

#### 2.1.3 Travail demandé

##### 2.1.3.1 *Programme A — Serveur vulnérable*

Écrire un programme en CLI (langage au choix) qui reçoit un mot de passe en paramètre et le compare à une valeur codée en dur dans le code. La comparaison doit être réalisée de façon naïve (caractère par caractère avec arrêt précoce), rendant le programme vulnérable à une attaque temporelle. Le programme retourne un code de retour différent selon que le mot de passe est correct ou non.

##### 2.1.3.2 *Programme B — Attaquant*

Écrire un programme qui, en appelant A de manière répétée et en mesurant précisément les temps de réponse, retrouve le mot de passe caractère par caractère, sans aucune connaissance préalable de sa valeur. Le programme doit afficher les caractères retrouvés au fur et à mesure.

#### 2.1.4 Rapport

- Expliquer le principe de l'attaque et les conditions qui la rendent possible
- Analyser les résultats obtenus (nombre d'appels nécessaires, fiabilité)
- Proposer et implémenter une correction (comparaison en temps constant) en justifiant son efficacité

### 2.2 Sujet 2 : Attaque par Padding Oracle sur AES-CBC

#### 2.2.1 Classification

Difficulté : Difficile

Catégorie : Cryptographie / Mathématique

## 2.2.2 Contexte

Le mode CBC (Cipher Block Chaining) utilisé avec un schéma de padding PKCS#7 est vulnérable à une attaque de type Padding Oracle lorsque le serveur renvoie une information distincte selon que le padding est valide ou non. Cette attaque, permet de déchiffrer un message sans connaître la clé.

## 2.2.3 Travail demandé

### 2.2.3.1 Programme A — Serveur Oracle

Implémenter un serveur local (socket TCP ou HTTP) qui :

- Chiffre un message confidentiel en AES-CBC avec une clé générée aléatoirement au démarrage (non accessible au client)
- Expose deux endpoints : un pour obtenir le chiffré initial, un pour soumettre un chiffré arbitraire et qui retourne uniquement `200 OK` ou `422 Padding Error`

### 2.2.3.2 Programme B — Attaquant

Implémenter l'attaque Padding Oracle complète qui, en interrogeant le serveur A de façon itérative, déchiffre le message bloc par bloc, octet par octet. Le programme doit afficher le message en clair retrouvé.

### 2.2.3.3 Rapport

- Expliquer le fonctionnement mathématique de l'attaque bloc par bloc
- Analyser la complexité de l'attaque (nombre de requêtes théoriques vs observés)
- Décrire les contre-mesures : mode AEAD (GCM), MAC-then-Encrypt vs Encrypt-then-MAC, réponses uniformes

## 2.3 Sujet 3 : Forge de signature RSA par faute

### 2.3.1 Classification

Difficulté : Très difficile

Catégorie : Cryptographie / Mathématique

### 2.3.2 Contexte

En 2006, Bleichenbacher a démontré qu'une vérification incorrecte du padding PKCS#1 v1.5 dans les signatures RSA (notamment dans certaines implémentations SSL/TLS) permettait de forger des signatures valides sans connaître la clé privée, en exploitant la malléabilité de RSA.

### 2.3.3 Travail demandé

#### 2.3.3.1 Programme A — Vérificateur vulnérable

Implémenter un vérificateur de signature RSA-PKCS#1 v1.5 qui effectue une vérification incorrecte du padding : au lieu de vérifier que le bloc de padding remplit exactement la structure attendue, il

vérifie uniquement que le début et la fin sont corrects (regex-style), laissant des octets non contrôlés. Utiliser une clé RSA de 2048 bits générée pour l'exercice.

### **2.3.3.2      *Programme B — Forge***

Implémenter l'attaque de Bleichenbacher : construire mathématiquement une valeur dont la racine cubique (dans le cas  $e=3$ ) ou dont une structure forgée donne un bloc qui passe la vérification défectueuse. Le programme produit une signature forgée pour un message arbitraire choisi par l'attaquant.

### **2.3.4          Rapport**

- Décrire la structure du padding PKCS#1 v1.5 et la faille exploitée
- Expliquer la construction mathématique de la forge
- Analyser quelles implémentations réelles ont été touchées (OpenSSL, NSS...)
- Proposer la correction : vérification stricte du padding, passage à PSS

## **2.4 Sujet 4 : Attaque par extension de longueur sur les fonctions de hachage**

### **2.4.1          Classification**

- Difficulté : Difficile
- Catégorie : Web / Cryptographie

### **2.4.2          Contexte**

Les constructions MAC naïves de la forme ``H(secret || message)`` utilisant des fonctions de la famille SHA (construites sur le paradigme Merkle-Damgård) sont vulnérables à une attaque par extension de longueur : un attaquant peut calculer ``H(secret || message || extension)`` sans connaître le secret, à condition de connaître la longueur de celui-ci.

### **2.4.3          Travail demandé**

#### **2.4.3.1      *Programme A — Serveur vulnérable***

Implémenter une API web minimale (HTTP) qui :

- Authentifie les requêtes par un MAC de la forme ``SHA256(secret || données_requête)``
- Accepte une requête si le MAC fourni est valide
- Implémente une action sensible (ex. : modification d'un paramètre ``admin=false``) vérifiée côté serveur

#### **2.4.3.2      *Programme B — Attaquant***

Sans connaître le secret (mais en connaissant sa longueur), étendre un message légitime intercepté pour y ajouter ``&admin=true`` et calculer le MAC valide correspondant. Implémenter l'extension de longueur manuellement en reconstituant l'état interne de SHA-256.

#### 2.4.4 Rapport

- Expliquer la construction Merkle-Damgård et pourquoi elle rend l'attaque possible
- Décrire l'implémentation de l'état interne reconstitué
- Expliquer pourquoi HMAC résiste à cette attaque (double hachage, ipad/opad)
- Tester avec différentes longueurs de secret inconnue (attaque par force brute sur la longueur)

## 2.5 Sujet 5 : Attaque par réutilisation de nonce sur AES-GCM

### 2.5.1 Classification

Difficulté : Très difficile

Catégorie : Cryptographie / Mathématique

### 2.5.2 Contexte

AES-GCM est un mode de chiffrement authentifié (AEAD) très répandu. Sa sécurité repose sur l'unicité du nonce (IV) pour une clé donnée. Si un même nonce est utilisé deux fois avec la même clé, l'attaquant peut récupérer le masque de chiffrement, déchiffrer les messages et forger des tags d'authentification valides.

### 2.5.3 Travail demandé

#### 2.5.3.1 *Programme A — Serveur vulnérable*

Implémenter un service de chiffrement qui chiffre des messages en AES-GCM-128 mais utilise un nonce déterministe ou réutilisé (ex : nonce incrémental tronqué, nonce fixe, générateur faible). Le service expose deux fonctions accessibles par un client : ``encrypt(message)`` et ``decrypt(ciphertext, tag, nonce)``

#### 2.5.3.2 *Programme B — Attaquant*

En obtenant deux chiffrés utilisant le même nonce et la même clé, l'attaquant doit :

1. Récupérer le XOR des deux plaintexts
2. Retrouver les plaintexts si l'un d'eux est partiellement connu
3. Forger un tag d'authentification valide pour un message arbitraire (attaque sur le champ polynomial de GCM)

### 2.5.4 Rapport

- Expliquer la construction mathématique d'AES-GCM (CTR + GHASH)
- Montrer pourquoi la réutilisation de nonce casse simultanément confidentialité et authenticité
- Analyser des cas réels (TLS 1.2, WPA2 KRACK)
- Proposer des stratégies de gestion de nonce robustes (nonce aléatoire 96 bits, compteur + détection de collision)

## 2.6 Sujet 6 : Implémentation et cassage de LFSR et générateurs pseudo-aléatoires faibles

### 2.6.1 Classification

Difficulté : Très difficile

Catégorie : Cryptographie / Mathématique

### 2.6.2 Contexte

Les générateurs de nombres pseudo-aléatoires (PRNG) sont au cœur de nombreux protocoles cryptographiques. Un PRNG cryptographiquement faible (LFSR non masqué, LCG, Mersenne Twister non conditionné) peut être reconstruit entièrement à partir d'un nombre limité de sorties observées, compromettant tous les secrets générés.

### 2.6.3 Travail demandé

#### 2.6.3.1 *Partie 1 — LFSR*

Implémenter un LFSR (Linear Feedback Shift Register) de degré  $n$  (ex. : 32 bits) avec un polynôme primitif. Écrire l'attaque par reconstruction algébrique : à partir de  $2n$  bits consécutifs de sortie, retrouver l'état initial complet et prédire les bits futurs.

#### 2.6.3.2 *Partie 2 — Mersenne Twister*

Implémenter ou utiliser l'implémentation standard de MT19937. Observer 624 sorties 32 bits consécutives et implémenter le clonage complet de l'état interne. Prédire les 100 prochaines sorties et les vérifier. Outils

#### 2.6.3.3 *Partie 3 — Application à un scénario réel*

Simuler un serveur de génération de tokens de session utilisant l'un de ces PRNG faibles. À partir de tokens observés (en tant que client légitime), prédire le token de session d'un autre utilisateur et simuler l'usurpation de session.

### 2.6.4 Rapport

- Formaliser l'attaque LFSR avec l'algèbre linéaire sur  $GF(2)$
- Décrire la structure de MT19937 et l'inversion de la fonction de tempering
- Analyser des cas réels (PHP rand(), Java Random, failles de générateurs embarqués)
- Décrire les propriétés d'un CSPRNG (ChaCha20, /dev/urandom) et pourquoi ils résistent

## 2.7 Sujet 7 : SSLStrip : déclassement HTTPS vers HTTP

### 2.7.1 Classification

Difficulté : Difficile

Catégorie : Web / Outils

## 2.7.2 Contexte

SSLStrip est une attaque publiée par Moxie Marlinspike en 2009. Placé en homme du milieu (MitM), un attaquant peut intercepter la navigation d'une victime et remplacer silencieusement tous les liens HTTPS par leurs équivalents HTTP, tout en maintenant lui-même une connexion HTTPS légitime avec le serveur cible. La victime communique en clair sans s'en rendre compte, et son navigateur n'affiche aucune alerte.

## 2.7.3 Mise en place de l'environnement

Monter un réseau isolé (machines virtuelles ou conteneurs Docker) composé de trois nœuds :

- **Victime** : navigateur sans HSTS préconfiguré (Firefox avec profil vierge)
- **Attaquant** : machine avec accès MitM (ARP spoofing ou passerelle par défaut contrôlée)
- **Serveur cible** : serveur web local exposant un formulaire de connexion en HTTPS, avec une redirection HTTP → HTTPS

## 2.7.4 Travail demandé

### 2.7.4.1 Phase 1 — Positionnement MitM

Mettre en place le positionnement MitM par ARP spoofing entre la victime et la passerelle. Vérifier que le trafic transite bien par la machine attaquante avec un outil d'analyse réseau.

### 2.7.4.2 Phase 2 — Proxy SSLStrip

Implémenter (déployer et configurer) un proxy transparent qui :

- Intercepte les requêtes HTTP de la victime
- Contacte le serveur cible en HTTPS pour son propre compte
- Renvoie la réponse à la victime en HTTP en remplaçant tous les `https://` par `http://` dans le HTML, les en-têtes `Location`, et les attributs `action` des formulaires
- Enregistre les informations saisis par la victime dans les formulaires (méthode POST)

### 2.7.4.3 Phase 3 — Analyse et contre-mesures

- Activer HSTS sur le serveur cible (`Strict-Transport-Security: max-age=...`) et constater que l'attaque échoue pour les visites ultérieures
- Ajouter le domaine à la liste HSTS Preload et observer la différence avec la première visite
- Tester l'impact de l'en-tête `Content-Security-Policy: upgrade-insecure-requests`

## 2.7.5 Rapport

- Expliquer pourquoi la redirection 301 ne suffit pas à protéger la première visite
- Décrire précisément ce que HSTS résout et ce qu'il ne résout pas (première visite, sous-domaines)
- Analyser les limites résiduelles : attaque sur les sous-domaines non couverts, NTP spoofing pour expirer le HSTS, DNS spoofing



- Documenter les captures réseau illustrant l'attaque et les contre-mesures

## 2.8 Sujet 8 — Chiffrement homomorphe simplifié

### 2.8.1 Classification

Difficulté : Très difficile

Catégorie : Cryptographie / Mathématique

### 2.8.2 Contexte

Le chiffrement homomorphe permet d'effectuer des calculs sur des données chiffrées sans jamais les déchiffrer. Un serveur peut ainsi traiter des données confidentielles sans y avoir accès. Ce sujet implémente un schéma partiellement homomorphe simplifié (addition et multiplication limitée), inspiré des constructions modernes mais sans leur complexité mathématique complète.

### 2.8.3 Travail demandé

#### 2.8.3.1 Partie 1 — Implémentation du schéma

Implémenter un schéma de chiffrement symétrique simplifié sur les entiers (variante jouet de LWE) :

- $q$  est un grand entier qui définit l'espace de travail
- $s$ , la clé secrète est un entier tiré aléatoirement entre 1 et  $q$
- $e$ , un bruit : un petit entier tiré aléatoirement entre -5 et +5

#### Chiffrement :

On tire au hasard un entier  $a$  entre 0 et  $q$ . On calcule :

$$b = (a \times s + e + m \times (q/2)) \text{ modulo } q$$

#### Déchiffrement :

On calcule :

$$v = (b - a \times s) \text{ mod } q$$

- Si  $v$  est proche de 0  $\rightarrow$  le bit était 0
- Si  $v$  est proche de  $q/2$   $\rightarrow$  le bit était 1

#### Travail demandé

- Implémenter `chiffrer(bit, s, q)` et `dechiffrer(a, b, s, q)`
- Tester avec différentes valeurs de  $q$  (1000, 10000, 65537)
- Tester avec différentes amplitudes de bruit ( $\pm 2$ ,  $\pm 10$ ,  $\pm 100$ )
- Tracer une courbe : à partir de quelle amplitude de bruit le déchiffrement commence-t-il à échouer ?

### 2.8.3.2 *Partie 2 — Opérations homomorphes*

C'est ici que la "magie" opère. On montre qu'on peut calculer **sans déchiffrer**.

#### Addition homomorphe

Si on a deux chiffrés :

- $(a_1, b_1)$  = chiffré de  $m_1$
- $(a_2, b_2)$  = chiffré de  $m_2$

Alors le chiffré de  $m_1 + m_2$  est simplement :

$(a_1 + a_2, b_1 + b_2)$  modulo  $q$

#### Pourquoi ça marche ?

$$\begin{aligned} b_1 + b_2 &= a_1 \times s + e_1 + m_1 \times (q/2) \\ &\quad + a_2 \times s + e_2 + m_2 \times (q/2) \\ &= (a_1 + a_2) \times s + (e_1 + e_2) + (m_1 + m_2) \times (q/2) \end{aligned}$$

C'est exactement la structure d'un chiffré de  $m_1 + m_2$ , avec un bruit  $e_1 + e_2$ .

Le bruit **s'accumule** à chaque addition.

#### Travail demandé

- `additionner(chiffré1, chiffré2, q) → nouveau chiffré`
- Vérifier que `dechiffrer(résultat)` donne bien  $m_1 + m_2$
- Enchaîner 10, 50, 100 additions et calculer à quel moment le bruit accumulé fait échouer le déchiffrement

### 2.8.3.3 *Partie 3 — Application et limites*

Scénario : 20 étudiants votent OUI (1) ou NON (0). Chacun envoie son vote chiffré au serveur. Le serveur additionne les 20 chiffrés — sans jamais voir les votes. Un tiers de confiance déchiffre uniquement le total.

Le serveur ne sait pas qui a voté quoi. Il sait seulement, à la fin, que par exemple 13 personnes ont voté OUI.

#### Travail demandé :

- Implémenter les 3 programmes : votants, serveur (addition), dépouillement (déchiffrement)
- Vérifier que le total déchiffré est correct
- Trouver expérimentalement le nombre maximum de votants avant que le bruit ne corrompe le résultat (dépend de  $q$  et de l'amplitude du bruit)

### 2.8.4 **Rapport**

- Le problème **LWE** : Expliquer intuitivement pourquoi  $a \times s + e$  est difficile à inverser sans  $s$
- Évolution du bruit : Graphe du bruit résiduel en fonction du nombre d'additions
- Compteur de votes : Code des 3 rôles, résultats expérimentaux, nombre max de votants

- Cas d'usage réels : Lister les cas d'usage connus du chiffrement homomorphe et pour chacun les limites de profondeur de calcul et de performances
- Comparaison : Expliquer les différences entre votre modèle (jouet de LWE) et ces deux modèles existants : Microsoft SEAL et OpenFHE

## 2.9 Sujet 9 : Attaque par dictionnaire et cassage de hash avec règles

### 2.9.1 Classification

- Difficulté : Normale
- Catégorie : Crypto / Outils

### 2.9.2 Contexte

Ce TP repose sur **Hashcat**

Il est accompagné de l'archive TP9\_Hash qui contient 2 fichiers :

- TP9\_Hash\_V1.txt
- TP9\_Hash\_V2.txt

La majorité des mots de passe réels ne sont pas aléatoires. Ils suivent des patterns humains prévisibles : Leetspeak, ajout d'années ou de chiffres en fin de chaîne, majuscule initiale, etc. Les outils modernes comme Hashcat exploitent ces patterns via des moteurs de règles qui transforment un dictionnaire de base en des millions de variantes candidates, rendant inefficace le stockage de mots de passe avec des fonctions de hachage inadaptées.

### 2.9.3 Travail demandé

#### 2.9.3.1 *Partie 1 : Analyse d'une fuite de base de données*

Le fichier TP9\_Hash\_V1.txt fourni contenant 500 hash MD5 et SHA-1 générés à partir de mots de passe tirés de fuites réelles connues :

- Identifier combien sont cassables avec le dictionnaire `rockyou.txt` sans transformation

#### 2.9.3.2 *Partie 2 : Moteur de règles*

Implémenter un générateur de règles minimal capable d'appliquer les transformations suivantes sur chaque mot du dictionnaire : capitalisation de la première lettre, ajout de suffixes numériques (0-99, année courante  $\pm 5$ ), substitutions leetspeak (`a` → 4, `e` → 3, `i` → 1, `o` → 0, `s` → 5), duplication (`password` → `passwordpassword`).

- Identifier combien sont cassables avec le dictionnaire `rockyou.txt` avec ces transformations

#### 2.9.3.3 *Partie 3 : Fonctions adaptées et comparaison*

Le fichier TP9\_Hash\_V2.txt fourni contenant 100 hash bcrypt et Argon2 générés à partir de mots de passe tirés de fuites réelles connues :

- Identifier combien sont cassables avec le dictionnaire `rockyou.txt` sans transformation
- Identifier combien sont cassables avec le dictionnaire `rockyou.txt` **avec** transformation
- Mesurer (ou estimer) le temps de cassage pour chacun des 4 algorithmes (MD5, SHA-1, bcrypt et Argon2) sous forme d'un tableau récapitulatif
  - Algorithme
  - % succès
  - Temps min
  - Temps médian
  - Temps Max

#### 2.9.4 Rapport

- Expliquer pourquoi MD5 et SHA1 sont inadaptés au stockage de mots de passe
- Décrire les propriétés de bcrypt et Argon2id (facteur de coût, mémoire, parallélisme)
- J'ai affaibli bcrypt & argon2 pour ce TP

```
# Paramètres volontairement affaiblis pour le TP
# EN PRODUCTION : bcrypt rounds>=12, argon2 memory>=65536
ph = PasswordHasher(
    time_cost=1, # affaibli (normal : 3)
    memory_cost=8, # 8 KB au lieu de 64 MB — attaquable par hashcat
    parallelism=1, # affaibli (normal : 4)
    hash_len=32
)
```

Expliquez le mécanisme de ces changements

- Proposer une politique de stockage de mots de passe conforme aux recommandations ANSSI/NIST 2024

## 2.10 Sujet 10 : Implémentation d'un protocole d'authentification Zero-Knowledge (ZKP)

### 2.10.1 Classification

- Difficulté : Difficile
- Catégorie : Mathématique / Protocole

### 2.10.2 Contexte

Une preuve à divulgation nulle de connaissance (Zero-Knowledge Proof) permet à un prouveur de convaincre un vérificateur qu'il connaît un secret, sans révéler la moindre information sur ce secret. Ce concept, fondamental en cryptographie moderne, est à la base de nombreuses applications : connexion anonyme, blockchain (zk-SNARKs), authentification sans mot de passe ...

## 2.10.3 Travail demandé

### 2.10.3.1 Partie 1 — Protocole de Schnorr interactif

Vous devez écrire 2 programmes qui communiquent via socket local :

Alice : connaît une clé privée secrète  $x$ , a publié  $Y = g^{**}x$

Bob : connaît uniquement  $Y$ , veut vérifier que Alice connaît  $x$

Les 3 étapes du protocole

#### Étape 1 — Alice s'engage (**commitment**)

Alice tire un nombre aléatoire  $r$  (jamais réutilisé !)

Alice calcule  $R = g^{**}r$

Alice envoie  $R$  à Bob

#### Étape 2 — Bob lance un défi (**challenge**)

Bob tire un nombre aléatoire  $c$

Bob envoie  $c$  à Alice

#### Étape 3 — Alice répond (**response**)

Alice calcule  $s = r + c \times x$

Alice envoie  $s$  à Bob

Bob vérifie :

$$g^{**}s == R \times Y^{**}c ?$$

Si oui → Alice connaît bien  $x$ .

Si non → elle ment (ou a fait une erreur).

Pourquoi ça marche ?

Si Alice ne connaît pas  $x$ , elle ne peut pas calculer  $s = r + c \times x$  correctement

Elle devrait deviner  $c$  avant de recevoir  $c$  — probabilité 1 sur beaucoup

Après 30 tours, la probabilité qu'un imposteur passe est négligeable

#### travail à faire

Écrire les 2 programmes Alice & Bob, qui communiquent par un socket

- Alice & Bob s'échangent  $R$ ,  $c$ ,  $s$  via socket
- Bob affiche "Alice est authentifiée" ou "Échec"
- Tester avec la vraie clé → succès en 1 tour
- Tester sans connaître  $x$  (valeur aléatoire) → échec immédiat

### 2.10.3.2 Partie 2 — Version non-interactive (Fiat-Shamir)

Le problème avec la version interactive :

Dans la vraie vie, Alice et Bob ne sont pas connectés en temps réel. On veut que Alice puisse produire une preuve seule, que n'importe qui peut vérifier plus tard (comme une signature numérique)

#### L'astuce de Fiat-Shamir

On remplace Bob (qui générait  $c$  aléatoirement) par une fonction de hachage :

$$c = \text{SHA256}(R \parallel \text{message})$$

Alice calcule elle-même son propre challenge en hachant son commitment et le message. Le résultat  $(R, s)$  est une preuve autonome jointe au message.

Avant (interactif) : Alice  $\rightarrow R \rightarrow$  Bob  $\rightarrow c \rightarrow$  Alice  $\rightarrow s \rightarrow$  Bob vérifie

Après (Fiat-Shamir) : Alice calcule  $R$ , puis  $c = \text{SHA256}(R \parallel \text{msg})$ , puis  $s$

Alice publie  $(\text{message}, R, s)$

N'importe qui vérifie :  $g^{**s} == R \times Y^{**\text{SHA256}(R \parallel \text{msg})}$

#### travail a faire

- Écrire le programme `signer(message, x)  $\rightarrow$  produit  $(R, s)$`
- Écrire le programme `verifier(message, Y, R, s)  $\rightarrow$  retourne vrai/faux`
- Tester qu'une signature valide sur `msg1` est rejetée pour `msg2` (anti-rejeu)

### 2.10.4 Partie 3 — Attaques et propriétés de sécurité

#### Attaque 1 — Rejeu

Scénario : un attaquant intercepte une preuve valide  $(R, s)$  pour le message "accès autorisé le 01/01/2026" et essaie de la réutiliser le 02/01/2026

#### travail a faire

- Montrer que la vérification échoue si le message change car  $c = \text{SHA256}(R \parallel \text{message})$  sera différent.

#### Attaque 2 — Simulation (propriété zero-knowledge)

C'est la partie la plus contre-intuitive. On veut montrer que la preuve ne révèle rien sur  $x$ .

Trick du simulateur : sans connaître  $x$ , on peut fabriquer des transcripts  $(R, c, s)$  qui semblent valides, MAIS seulement si on choisit  $c$  et  $s$  d'abord, puis on calcule  $R = g^{**s} / Y^{**c}$  "à l'envers"

#### travail a faire

- Écrire un programme `simuler()`  $\rightarrow$  produit un faux transcript  $(R, c, s)$  qui passe la vérification
- Montrer que ce transcript est "indiscernable" d'un vrai

Conclusion : si même un simulateur sans connaissance de  $x$  peut produire des transcripts valides, c'est bien la preuve que les vrais transcripts ne révèlent rien sur  $x$

### **Attaque 3 — Réutilisation du nonce $r$**

C'est le point critique du TP : Si Alice utilise le même  $r$  deux fois pour deux challenges  $c1$  et  $c2$  :

$$s1 = r + c1 \times x$$

$$s2 = r + c2 \times x$$

Alors

$$s1 - s2 = (c1 - c2) \times x$$

$$x = (s1 - s2) / (c1 - c2)$$

Conclusion :

La clé privée est compromise en une soustraction.

#### **travail a faire**

Simuler le cas dégénéré (même  $r$  forcé deux fois)

Récupérer  $x$  effectivement

## **2.11 Sujet 11 : Certificate Pinning et attaque MITM sur application mobile**

### **2.11.1 Classification**

- Difficulté : Difficile
- Catégorie : Système / Outils

### **2.11.2 Contexte**

Les applications mobiles qui se contentent de valider la chaîne de certification standard sont vulnérables à un attaquant capable d'installer un certificat CA racine de confiance sur l'appareil (proxy d'interception, MDM d'entreprise, utilisateur root). Le certificate pinning est une défense qui fixe dans l'application l'empreinte du certificat attendu, rendant ce type d'interception inefficace ... jusqu'à ce que l'application elle-même soit modifiée.

### **2.11.3 Travail demandé**

#### **2.11.3.1 Partie 1 — Application vulnérable**

Développer une application cliente simple (Python avec `requests`, ou application Android minimaliste) qui consomme une API HTTPS (serveur local) en validant uniquement la chaîne de certification standard. Mettre en place un proxy d'interception (Burp Suite, mitmproxy, ...) avec son CA installé sur le système de test, et démontrer l'interception complète du trafic.

### **2.11.3.2    *Partie 2 — Mise en place du pinning***

- Ajouter le certificate pinning dans l'application cliente : vérifier que l'empreinte SHA-256 du certificat serveur (ou de sa clé publique) correspond à une valeur codée dans l'application
- Vérifier que l'interception par proxy échoue désormais avec une erreur explicite
- Implémenter un mécanisme de backup pin (second certificat de secours) pour simuler la rotation de certificat sans casser l'application

### **2.11.3.3    *Partie 3 — Contournement du pinning***

Démontrer deux techniques de contournement sur l'application cliente :

**Modification statique** : patcher le binaire ou le code source pour remplacer l'empreinte pinnée par celle du certificat du proxy

**Hook dynamique** : utiliser Frida pour hooker la fonction de vérification SSL au runtime et la faire retourner systématiquement "valide" sans modifier le binaire

### **2.11.4      *Rapport***

- Expliquer la différence entre certificate pinning et public key pinning, et leurs avantages respectifs
- Analyser les limites du pinning comme mécanisme de sécurité (dépendance à l'intégrité du binaire)
- Décrire les bonnes pratiques de déploiement : durée de vie des pins, backup pins, mécanisme de mise à jour
- Discuter des alternatives : Certificate Transparency, DANE/TLSA, mutual TLS (mTLS)Outils

## **2.12      *Sujet 12 : Analyse et exploitation d'une mauvaise gestion des JWT***

### **2.12.1      *Classification***

- Difficulté : Normale
- Catégorie : Web / Outils

### **2.12.2      *Contexte***

Les JSON Web Tokens sont omniprésents dans les architectures modernes d'authentification. Leur flexibilité est aussi leur principal danger : de nombreuses implémentations ont souffert de vulnérabilités critiques liées à une vérification incorrecte de l'algorithme de signature, permettant à un attaquant de forger des tokens d'authentification arbitraires sans connaître le secret.



## 2.12.3 Travail demandé

### 2.12.3.1 Partie 1 — Mise en place d'une API avec JWT

Implémenter une API REST avec trois niveaux d'accès : public, utilisateur authentifié, administrateur. L'authentification repose sur des JWT signés avec HMAC-SHA256 (HS256). Les endpoints sensibles vérifient la signature et le claim `role` dans le payload.

### 2.12.3.2 Partie 2 — Exploitation des vulnérabilités classiques

Écrire des programmes qui démontrent les attaques suivantes :

- **Attaque `alg:none`** : modifier le header du JWT pour spécifier `"alg": "none"` et supprimer la signature. Si le serveur accepte cet algorithme, n'importe quel payload est accepté.
- **Confusion RS256/HS256** : si le serveur expose sa clé publique RSA et qu'une version vulnérable accepte un token HS256 signé avec cette clé publique (en la traitant comme un secret HMAC), forger un token administrateur.
- **Brute-force du secret HS256** : à partir d'un token valide intercepté, retrouver un secret HS256 faible (< 8 caractères) par attaque dictionnaire.

### 2.12.3.3 Partie 3 — Durcissement

Dans une branche Git distincte : Corriger l'API pour résister aux trois attaques précédentes, puis implémenter :

- Validation explicite de l'algorithme attendu (whitelist, jamais `alg:none`)
- Rotation des secrets / paires de clés avec `kid` (Key ID) dans le header
- Courte durée de vie des tokens (`exp`) et mécanisme de révocation par liste noire en base

## 2.12.4 Rapport

- Expliquer la structure d'un JWT (header, payload, signature) et le flux d'authentification complet
- Détailler chaque vulnérabilité exploitée et la condition côté serveur qui la rend possible
- Comparer les approches HS256 (symétrique) et RS256/ES256 (asymétrique) selon les architectures (monolithique, microservices)
- Référencer les recommandations OWASP sur la gestion des tokens de session